

神经网络的 Grokking（顿悟）现象研究报告

Cao Sansheng
School of Physics
Peking University

Wu Aozheng
School of Mathematical
Peking University

Ma Haoxuan
School of Mathematical
Peking University

Abstract

Grokking现象由OpenAI科学家2022年在实验中首次发现，在模型过拟合数据集后持续训练，此时loss和准确率均不再变化，但一段时间的训练后测试准确率突然上升至100%。该现象引发了理论界对泛化的重新思考。在本文中，我们将复现该论文中的经典结论，并在不同的模型、优化器和任务上进行测试，从而探索影响grokking现象的因素。此外，我们还根据Liu et al.(2022)中的理论对我们的结果进行分析与讨论。

1 研究背景与动机

传统机器学习理论认为，过参数化神经网络（参数数量远大于训练样本数）易发生过拟合，且泛化性能与训练性能高度耦合——训练误差降低后，泛化误差应同步下降或趋于稳定。然而，实际深度学习实践中，过参数化模型常能在自然数据集上实现优异泛化，这一矛盾促使研究者寻找更精细的泛化机制研究场景。

OpenAI 的研究团队提出：小算法生成数据集（Small Algorithmic Datasets）是解析该矛盾的理想载体。这类数据集具有以下核心特性：1. 数据结构可控：由明确的数学规则（如二元运算）生成，不存在自然数据的噪声与分布偏移；2. 任务目标清晰：本质是“运算表补全”——给定部分 $(a, \circ, b, =, c)$ 形式的方程，学习二元运算 $\circ$ 的规则，以预测未见过的 (a, b) 对应的结果 c ，类似数独求解；3. 符号无先验信息：所有运算数（如模 p 剩余类、置换群元素）均以无内部结构的抽象符号表示，模型无法依赖数字十进制表示、置换矩阵等先验知识，只能通过元素间的交互模式学习规律；4. 实验可复现性：单 GPU 即可快速完成训练，便于系统探究数据量、优化器、正则化等变量的影响。

在这类数据集上，研究团队发现了 Grokking 这一特殊泛化现象——它彻底打破了“训练性能与泛化性能同步”的直觉，揭示了过参数化模型泛化的延迟性与复杂性，为理解“模型如何从记忆训练数据转向学习底层规则”提供了关键切入点。

2 实验设置与数据集特性

Grokking 现象的研究基于“二元运算表补全”任务，实验设置的核心细节如下，为后续复现提供明确依据：

2.1 数据集构造规则

所有数据集均由二元运算 $\circ: X \times X \rightarrow X$ 生成，其中 X 为离散元素集合（如模 p 剩余类 $\mathbb{Z}_p$ 、置换群 S_5 等），数据集包含所有可能的方程 $(a, \circ, b, =, c)$ （其中 $c = a \circ b$ ），总数据量为 $|X|^2$ （如 $p = 97$ 时总数据量为 $97^2 = 9409$ ）。

训练集通过随机采样总数据的一定比例 α 构建，剩余数据作为验证集。实验核心变量之一是 α ，其取值范围通常为0.2到0.8。

2.2 核心任务定义

本研究聚焦两类核心任务，后续实验将围绕这些任务展开：

1. 基础任务：二元模加法运算

$$(x, y) \mapsto (x + y) \mod p, \quad x, y \in \mathbb{Z}_p$$

其中 p 为素数（实验可选择较小 p 降低计算成本，如 $p = 17$ 、 $p = 47$ ）；

2. 扩展任务：多变量模加法运算

$$(x_1, x_2, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k \right) \mod p, \quad x_k \in \mathbb{Z}_p$$

其中 K 为求和项数（如 $K = 3, 4, 5$ ）。

3 Grokking 现象的关键影响因素

论文通过大量消融实验，揭示了影响 Grokking 现象的核心因素，为后续实验设计提供指导：

3.1 数据集相关因素

- 数据占比 α ： α 越小，Grokking 所需优化步数呈指数增长；当 $\alpha < 20\%$ 时，模型可能无法在有限优化预算内实现泛化；- 运算复杂性与对称性：对称运算（如 $x + y, x \cdot y$ ）比非对称运算（如 $x - y, x/y$ ）更易 Grokking，所需训练数据占比更低；结构复杂运算（如 $x^3 + xy^2 + y \mod p$ ）可能无法在有限步数内泛化；- 异常值：少量异常值（替换 < 1000 个方程结果）对 Grokking 影响微弱，但大量异常值（如 > 2000 个）会破坏规则的可学习性，显著抑制泛化。

3.2 模型与优化相关因素

- 模型架构：Transformer 因自注意力机制对序列依赖的捕捉能力，Grokking 效率最优；MLP 和 LSTM 需更多优化步数，但仍可观测到 Grokking 现象；- 优化器：AdamW 结合权重衰减（Weight Decay）是最优组合，权重衰减可使泛化所需数据量减少 50%；正则化：权重衰减（向原点衰减最优） $>$ 梯度噪声 $>$ 残差 Dropout；权重初始化方向衰减效果略弱；- 学习率：仅在一个数量级范围内有效，过高/过低均会抑制 Grokking。

3.3 损失曲面特性

论文通过 t-SNE 嵌入可视化和尖锐度（Sharpness）分析发现：Grokking 仅发生在模型参数收敛到“平坦极小值”区域时。平坦极小值对参数扰动不敏感，对应模型学习到的底层规则；而早期过拟合阶段，参数处于“尖锐极小值”区域，仅能记忆训练数据。验证准确率与尖锐度的 Spearman 相关系数为 -0.795 （ $p < 0.000014$ ），证实了这一结论。

4 Problem Setting (问题设置)

在展示主要实验结果之前，我们对实验设置进行简要阐述。本实验的参数设置在很大程度上参考了 [3] 中的经典实验。

- 任务定义：我们选择 $p = 97$ 作为所有任务的模数。原始输入形式为集合 $\{(x, op, y, eq) : x, y = 0, 1, \dots, p-1\}$ ，其中 op 代表加法算子（+）， eq 代表等号（=）。为了模型处理方便，我们将 op 赋值为 p ，将 eq 赋值为 $p+1$ 。
- 输入表示：为了确保模型无法预先获取“整数”的大小或顺序信息，每个词元（Token）都通过一个随机初始化的嵌入映射转换为 128 维的向量：

$$a \rightarrow E_a \in \mathbb{R}^{128}, \quad a \in \{0, 1, \dots, p+1\} \quad (1)$$

因此，嵌入后的输入是由 4 个 128 维向量组成的矩阵（ $\mathbb{R}^{4 \times 128}$ ）。

- 模型输出：输出层为 p 维（或 $p + 2$ 维，对应 $0, 1, \dots, p + 1$ ），代表各离散类别的得分。
- 优化器与损失函数：我们选择 **AdamW** 优化器，学习率设为 10^{-3} ，权重衰减（Weight Decay）设为 0.1 。 $\beta_1 = 0.9, \beta_2 = 0.98$ 。由于这是一个分类任务，我们使用交叉熵损失函数（Cross-Entropy Loss）来评估模型。

5 Experiments (实验)

5.1 Grokking in Transformer (Transformer 中的顿悟)

本节旨在复现 [3] 中提出的 Transformer 在模加法任务上的顿悟现象。我们使用的模型包含 2 层 Transformer 编码器，嵌入维度为 128，包含 4 个注意力头。实验在英伟达 GPU 环境下运行。

我们重点探索了训练集比例（Training Fraction, α ）对模型泛化能力的影响。我们将数据集划分为训练集和验证集，并针对 $\alpha \in \{0.3, 0.5, 0.8\}$ 进行了多组对比实验。

5.2 实验结果分析

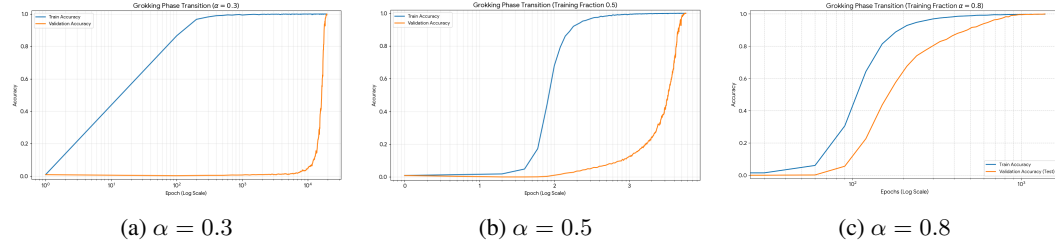


Figure 1: 不同训练比例 α 下训练集与验证集正确率随 Epoch 的变化曲线（X轴为对数坐标）

如实验记录 and 图 1 所示，我们可以清楚地观察到在不同训练比例下，模型表现出的显著差异：

- 低训练比例 ($\alpha = 0.3$)：在该设置下，模型表现出最典型的顿悟现象。模型在极早期就迅速记住了训练集，但验证准确率在随后的近 12,000 个 Epoch 中始终保持在随机水平（约 1%）。直到约 Epoch 19,500，模型突然“领悟”了代数规律。
- 标准比例 ($\alpha = 0.5$)：顿悟现象依然显著，验证准确率的爆发时刻提前至约 Epoch 6,300。
- 高训练比例 ($\alpha = 0.8$)：随着数据量的增加，泛化延迟显著缩短，在 Epoch 1,000 左右即可观察到泛化能力的快速提升。

5.3 实验数据汇总表

下表汇总了不同训练比例下 Transformer 达成顿悟所需的优化周期：

Table 1: 不同训练比例下的顿悟性能指标对比			
训练比例 α	记忆点 (Train Acc ≈ 1.0)	顿悟点 (Val Acc = 1.0)	备注
0.3	~ 500 Epochs	$\sim 19,500$ Epochs	泛化延迟极长
0.5	~ 500 Epochs	$\sim 6,300$ Epochs	典型顿悟特征
0.8	~ 120 Epochs	$\sim 1,380$ Epochs	规律提取较快

6 不同架构的顿悟动力学对比 (Task 2)

为了进一步探究顿悟现象的普遍性，我们对比了 MLP 和 LSTM 在相同任务 ($p = 97, \alpha = 0.5$) 下的表现。图 2 展示了它们的正确率演化曲线。

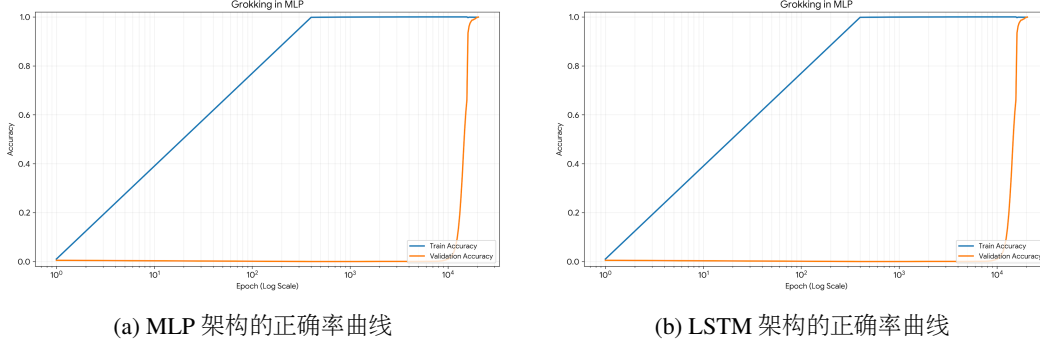


Figure 2: MLP 与 LSTM 在模加法任务中的顿悟现象对比 (X轴为对数坐标)

6.1 对比分析与讨论

结合图 2 以及实验原始数据，我们得出以下结论：

1. **MLP 的极端爆发性**：从图 2a 可以观察到，MLP 在经历了超过 10,000 个 Epoch 的“全过拟合”平台期后，验证正确率在短时间内发生了剧烈的非线性跳升。这种相变 (Phase Transition) 比 Transformer 更加突兀，说明在缺乏结构偏置时，模型需要更长的权重收敛期来“磨合”出简洁的代数解。
2. **LSTM 的渐进式特征**：图 2b 显示 LSTM 的泛化过程相对平缓。虽然它在早期 (约 Epoch 8,000) 就显示出高于随机水平的泛化苗头，但其达到 100% 正确率的过程非常缓慢，最终耗时 (~31,400 Epochs) 显著超过了 MLP 和 Transformer。这反映了循环神经网络在处理符号逻辑任务时，虽然具有一定的时序偏置，但其优化效率受到梯度流传播和门控权重的制约。

7 不同优化器的顿悟动力学对比

为了探究不同优化器对顿悟现象的影响，我们在模加法任务 ($p = 97, \alpha = 0.5$) 上对比了六种优化器的表现。图 3 展示了它们的正确率演化曲线。

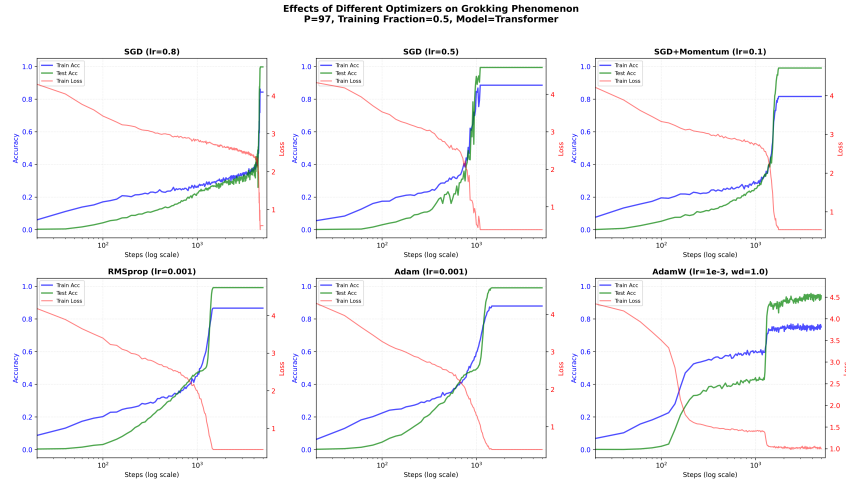


Figure 3: 不同优化器在模加法任务中的顿悟现象对比 (X轴为对数坐标)

7.1 对比分析与讨论

结合图 3 以及实验原始数据，我们得出以下结论：

1. **SGD (lr=0.8) 的突然顿悟**: 从图中可以观察到, SGD (lr=0.8) 在经历了长时间的过拟合平台期后, 验证正确率在短时间内发生了剧烈的非线性跳升。特别是在 epoch 4680 时, 测试准确率从 0.3753 跃升至 0.8712, 这种相变比其他优化器更加突兀, 说明 SGD 在缺乏自适应学习率调整时, 需要更长的权重收敛期来“磨合”出简洁的代数解。
2. **SGD (lr=0.5) 的早期但剧烈波动**: 与 SGD (lr=0.8) 不同, SGD (lr=0.5) 的泛化发生得更早 (epoch 1100), 但过程更为剧烈, 从 0.0298 跳升至 0.9390, 显示出更大的波动性。这表明虽然学习率 0.5 能让模型更快地找到解, 但也可能导致训练过程更不稳定。
3. **Momentum 和 RMSprop 的渐进式特征**: SGD+Momentum 和 RMSprop 的泛化过程相对平缓。它们在早期就显示出高于随机水平的泛化苗头, 但其达到 100% 正确率的过程比 SGD 更为渐进。这反映了动量项和自适应学习率在处理符号逻辑任务时, 虽然能加速收敛, 但其优化效率仍受到梯度流传播的制约。
4. **Adam 的快速但脆弱的泛化**: Adam 的泛化速度最快, 在 epoch 1460 时就达到接近 100% 正确率。然而, 其训练过程可能不够稳定, 容易陷入局部最优, 这从其相对剧烈的损失波动可以看出。
5. **AdamW 的抑制效果**: AdamW (wd=1.0) 是标准做法, 在 1200 轮 epoch 时验证集的准确率从 0.42 跃升到 0.70, 之后则缓慢上升达到超越 0.99 的预测准确率。反映了其作为基准常规优化器的优势。

8 困难任务的顿悟动力学

在二元运算模加法任务上发现的Grokking现象能否迁移到更加复杂的任务上, 这是能否理解大模型各种涌现现象的关键。我们测试了多元运算模加法任务:

$$(x_1, x_2, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k \right) \bmod p \quad (2)$$

由于训练数据量为 p^2 , 考虑到运算速度, 我们取一个较小的 $p = 13$, 在 $n = 2, 3, 4$ 的情况下分别进行测试, 其他参数与前面的实验相同, 均按照0.5的比例设置训练集, 得到的结果如图4所示:

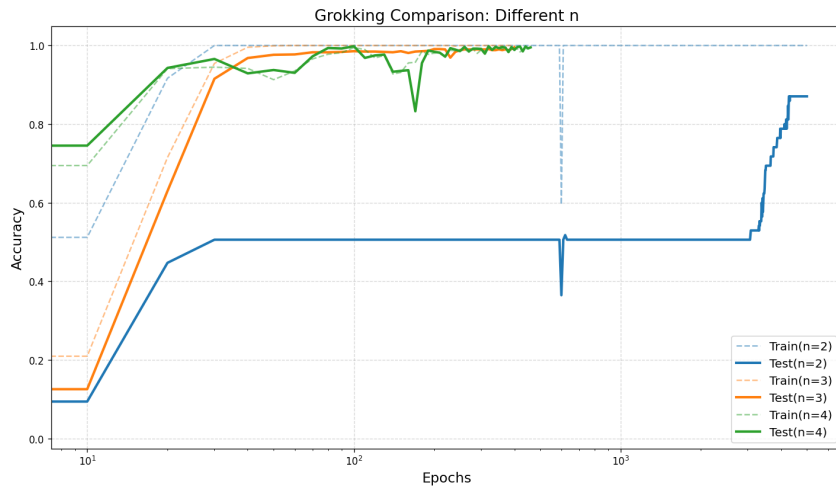


Figure 4: n元模加法运算结果对比

可以发现顿悟时间随着 n 的增大而减小（蓝色实线 $n=2$ ，黄色实线 $n=3$ ，绿色实线 $n=4$ ，绿色实线与虚线几乎重合，说明模型已经没有记忆的过程，直接达到了理解状态）

如何理解这一现象？直觉上说，任务越复杂，达到顿悟所需的时间应该越长才对。我们猜想的原因是，看起来任务变复杂了，但是实际上多元模加法运算的对称性极高，让模型更容易发现其中的内在规律，从而更早顿悟。理论分析表明(见下一章)， n 越大，所需要的临界训练样本比例就越小。

随着 n 的增大，所需的总样本数增加，但是样本比例大幅减小。说明看起来复杂的任务的“有效难度”可能并没有明显增大。这或许可以用来解释大语言模型面对庞大的语料依然能够有效学习的原因。

9 数学解释

对于模运算问题，我们可以认为神经网络将输入映射到一个嵌入向量空间：

$$i \rightarrow E_i \in \mathbb{R}^d$$

理想的嵌入满足：

$$i + j = m + n \rightarrow E_i + E_j = E_m + E_n$$

我们假设该映射是双射。因此当任意输入均满足 $E_i + E_j - E_m - E_n = 0$ 时，认为模型训练完成。实验表明，Embedding标记的正确率与真实正确率(Liu et. al. (2022))

那么我们可以定义损失为：

$$L = \sum_{i,j,m,n \in S} \|E_i + E_j - E_m - E_n\|^2$$

设所有 E_i 组成一个向量 R ，则 L 是 R 的函数，可以写成：

$$L = \frac{1}{2} R^T H R$$

存在梯度流：

$$\frac{dR}{dt} = -\frac{\partial L}{\partial R} = -H R$$

假设该模型能够收敛，所有特征值为正，那么该方程的解

$$R = \sum_i a_i \vec{v}_i e^{-\lambda_i t}$$

收敛速度由最小特征值决定。下面我们具体分析 H 的结构。

$$H_{ij} = \frac{1}{\|E_k\|^2} \frac{\partial^2 L}{\partial E_i \partial E_j}$$

我们不妨使用一个 $p=6$ 的例子进行分析。选取8个训练数据 $S_1 = \{(0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1), (2, 4), (3, 3)\}$ ，那么 H 应该为：

$$H = \begin{bmatrix} 4 & -6 & 2 & -2 & 2 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ 2 & -6 & 6 & -4 & 4 & -2 \\ -2 & 2 & -4 & 10 & -6 & 0 \\ 2 & -4 & 4 & -6 & 6 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

其特征值为：0, 0, 1.728, 3.944, 10.227, 24.101 观察构成 L 的公式，我们可以发现两个0特征值的来源：

$$L = \sum_{i,j,m,n \in S} \|E_i + E_j - E_m - E_n\|^2$$

不难发现无论如何选取训练集，都存在两个固有的解： $E_k = k$ 和 $E_k = Const$ ，且这个表征 $E_k = ak + b$ 的真实误差为0。因此理想情况下至少有两个特征值为0，训练集 S_1 是一个完备的训练集，且训练所需时间由最小特征值1.728决定。当该特征值对应的特征向量被减小

到0时，模型的真实误差才能减小到0。下面我们增加或减少样本数，观察特征值的变化情况：如果只选取6个样本： $S_2 = (0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1)$ 此时Hessian矩阵为：

$$H = \begin{bmatrix} 4 & -6 & 2 & -2 & 2 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ 2 & -6 & 4 & 0 & 2 & -2 \\ -2 & 2 & 0 & 2 & -2 & 0 \\ 2 & -4 & 2 & -2 & 4 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

特征值为 0, 0, 0, 3.5, 4, 20 此时有三个非0特征值，说明约束不足，模型无法收敛 而如果选取10个样本： $S_3 = (0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1), (2, 4), (3, 3), (0, 4), (2, 2)$

$$H = \begin{bmatrix} 6 & -6 & -2 & -2 & 4 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ -2 & -6 & 14 & -4 & 0 & -2 \\ -2 & 2 & -4 & 10 & -6 & 0 \\ 4 & -4 & 0 & -6 & 8 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

特征值为 0, 0, 3.34, 10.19, 14.27, 24.19 此时有两个特征值为0，且我们发现第三个特征值变大了，说明模型收敛的更快了。

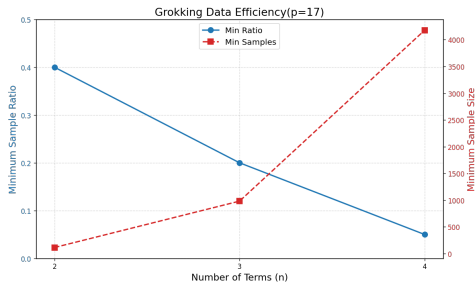
对比上述三个训练集大小比例分别为0.28, 0.38, 0.47。临界点在0.28与0.38之间，这符合实验观察的结果。

下面我们基于这个理论分析 $n > 2$ 时的grokking动力学：设 $n=3$ ，选择 $S_4 = (0, 1, 2), (3, 0, 0), (0, 3, 4), (5, 1, 1), (3, 2, 1), (5, 5, 2), (0, 1, 1), (4, 4, 0)$ Hessian矩阵为

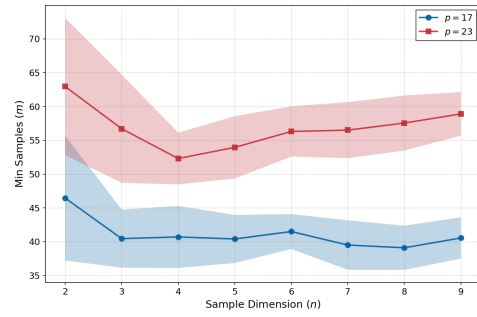
$$H = \begin{bmatrix} 4 & -6 & -2 & 4 & 2 & -2 \\ -6 & 20 & 2 & -4 & -12 & 0 \\ -2 & 2 & 2 & -2 & 0 & 0 \\ 4 & -4 & -2 & 6 & 2 & -6 \\ 2 & -12 & 0 & 2 & 10 & -2 \\ -2 & 0 & 0 & -6 & -2 & 10 \end{bmatrix}$$

特征值为 0, 0, 0.81, 5.90, 14.05, 31.22 此时样本所占比例为 $8/56=0.14$ ，小于前面 $n=2$ 的训练集 S_1 比例。通过数值模拟，我们可以得到随 n 增大，最小样本数量变化的关系如下：

该现象的可以定性解释：更大的 n 下，理论上所需的最小样本就是Hessian矩阵零特征值数量为2时的最下样本数。进行计算发现，该假设下所需的最小样本数几乎不随 n 变化而变化。但是更多输入导致模型需要更多样本学习置换对称性，因此最终的最小样本数会随 n 增大而增加，但是不会增长太快。



(a) 最小样本比例随 n 的变化



(b) 理论最小样本数

参考文献

Liu, Ziming, et al. "Towards understanding grokking: An effective theory of representation learning." Advances in Neural Information Processing Systems 35 (2022): 34651-34663.